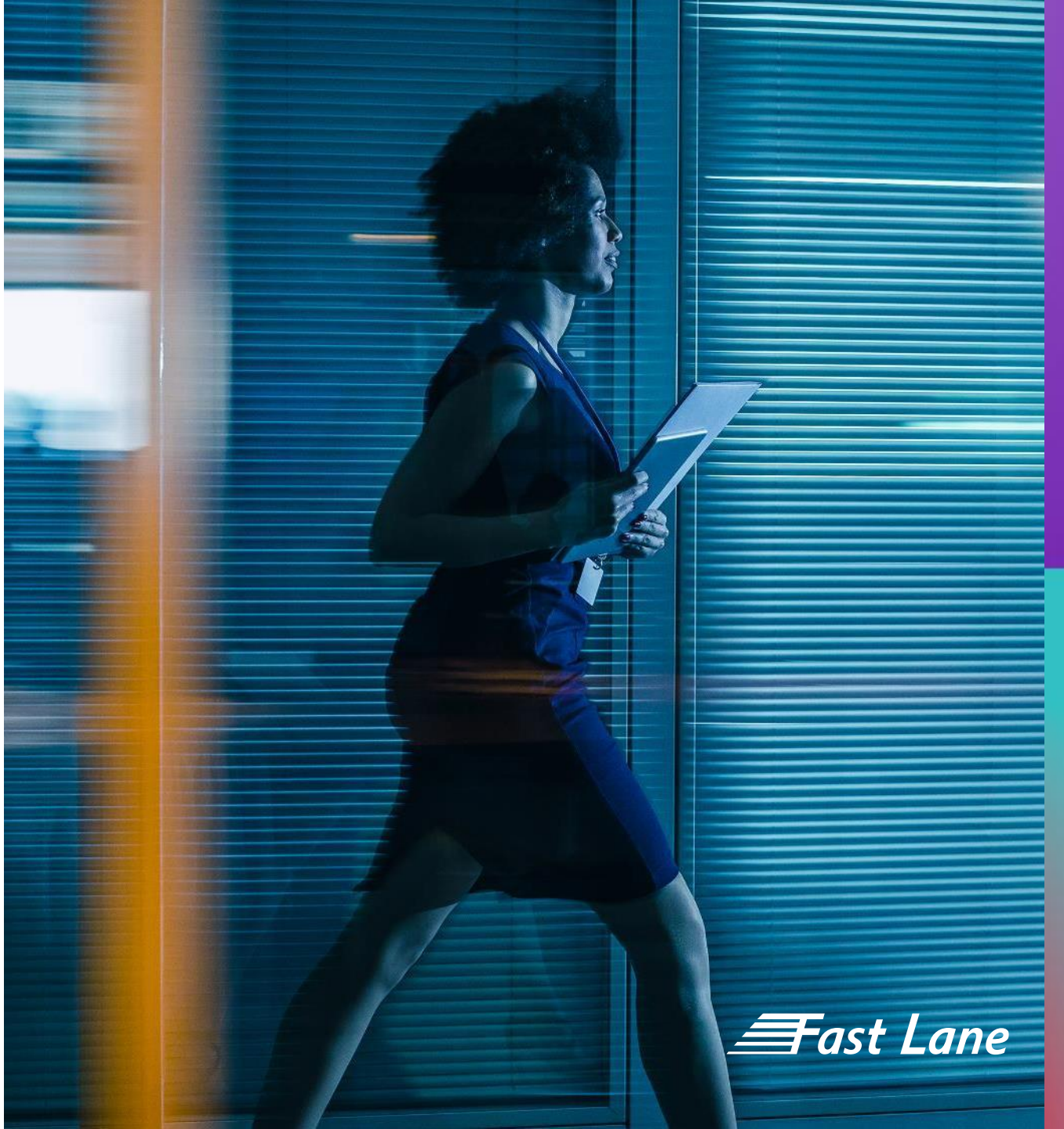
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 3

Creating & Managing Skills

Folders Claude loads when the work matches



01

Intro

Skills, slash commands, anatomy and installing skills

02

Creating

SKILL.md, frontmatter and a skill from a chat

03

Composite

Scripts, references and composing skills

04

Live Data and Subagents

Injected command output and forked skills

05

Test and Package

Trigger tests, four layers and shipping

Creating and managing skills

Folders Claude loads when the work matches

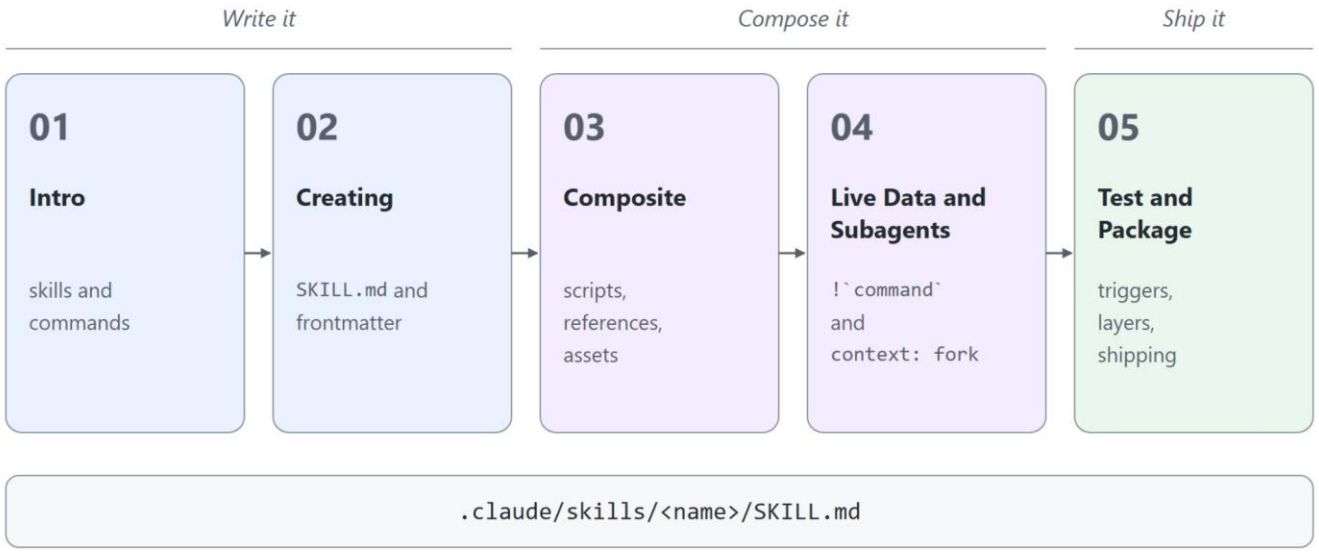
- **Authoring**: anatomy, frontmatter and a skill extracted from a proven chat
- **Composition**: scripts, references, live data and forked subagents
- **Shipping**: trigger tests, the four layers and packaging for strangers



Overview

The path through skills

- Start with how a skill differs from a slash command
- Write **SKILL.md**: placement, frontmatter and the instruction body
- Bundle **scripts** and **references**, then compose skills
- Inject live data with `` !`command` ``, fork into a **subagent**
- Finish by **testing** triggers and packaging the folder



01

Intro

Skills, slash commands, anatomy and installing skills

Skills and slash commands

A folder or a single file

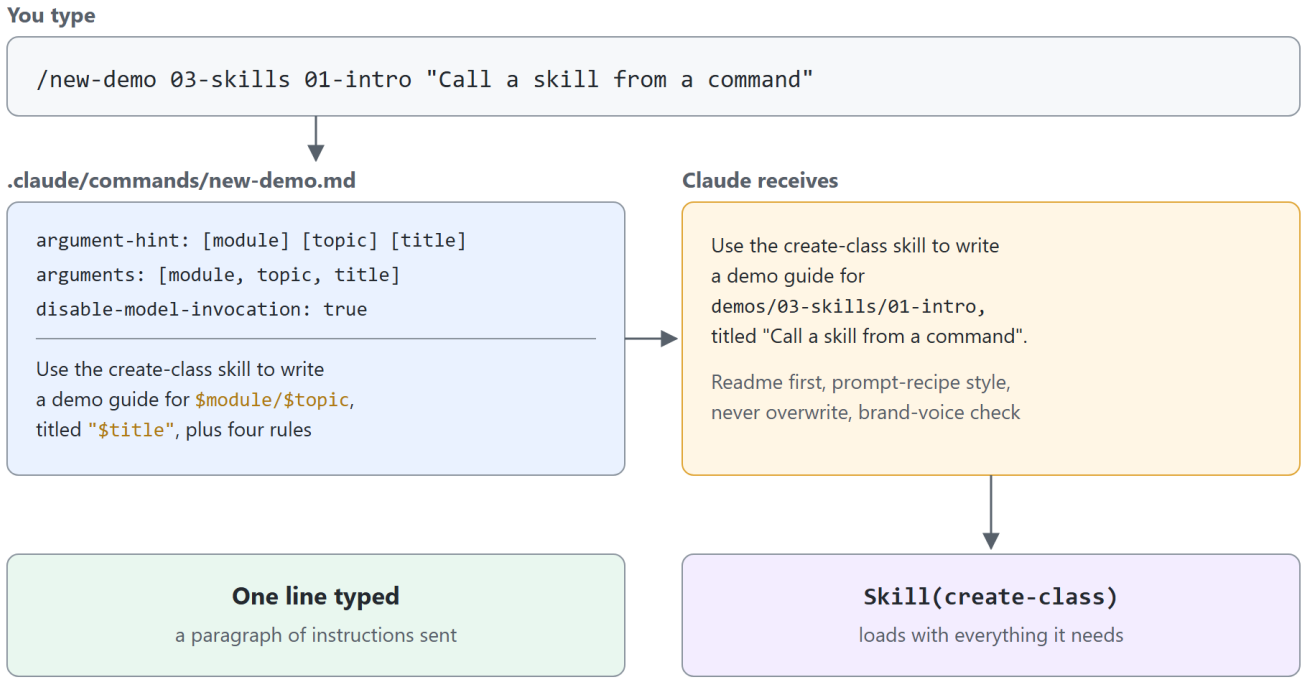
- Both give you a `/name` and accept the same frontmatter
- A **skill** is a folder that can ship references, scripts and templates
- A **command** is one file; everything must fit inside it
- `description` and `disable-model-invocation` control triggering in both
- Claude-triggered runs get no arguments: `$ARGUMENTS` stays empty



A short command that calls a skill

The command holds the paragraph, you type the values

- Calling a skill well takes a **paragraph**: skill, folder, style, checks
- A **command** carries that paragraph once and names the skill
- `arguments` turns each value you type into a `$name`
- `argument-hint` shows the arguments in the `/` menu
- `disable-model-invocation`: only you run it, with arguments



Skill anatomy

Frontmatter that routes, a body that instructs

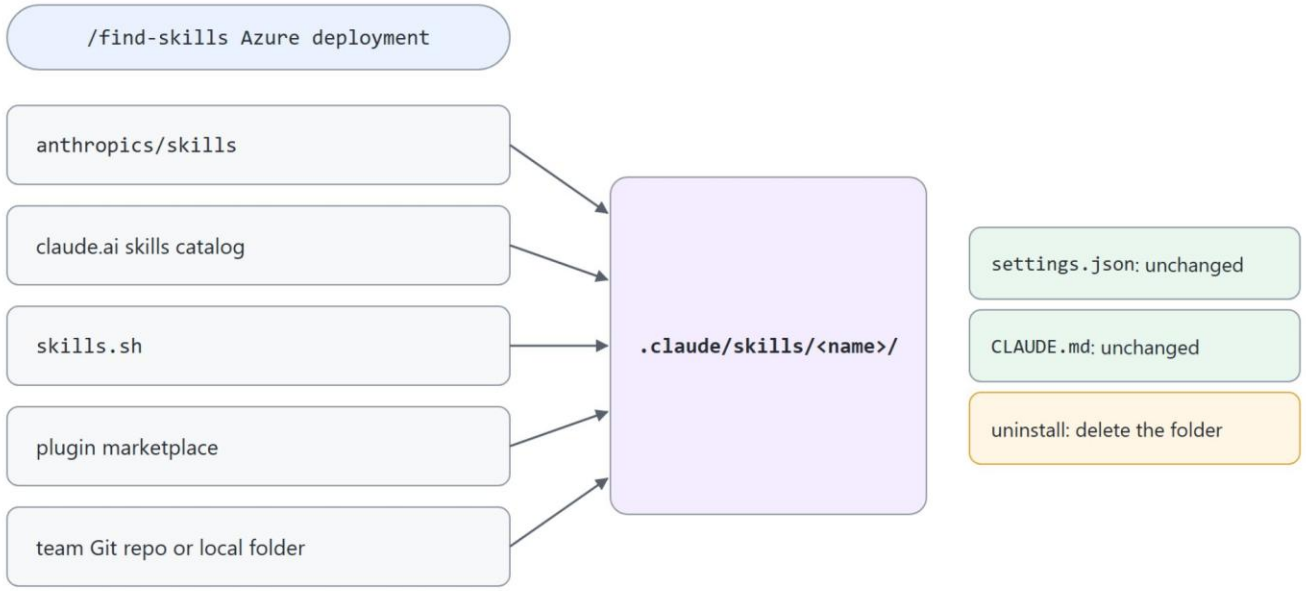
- YAML frontmatter, then a Markdown body: **Inputs**, **Steps**, **Output Format**
- `description` carries the trigger phrases users actually type
- `allowed-tools` pre-approves tools; `disallowed-tools` takes them away
- The transcript line `Skill(<name>)` confirms the skill loaded



Installing skills

Catalogs, /find-skills and what an install changes

- `/find-skills` searches the catalog from inside a session
- Sources: **anthropics/skills**, the claude.ai catalog, **skills.sh**, plugin marketplaces
- Team skills install from a private Git repo or a local folder
- An install **drops a folder** and changes nothing else
- Deleting the directory is the entire uninstall



02

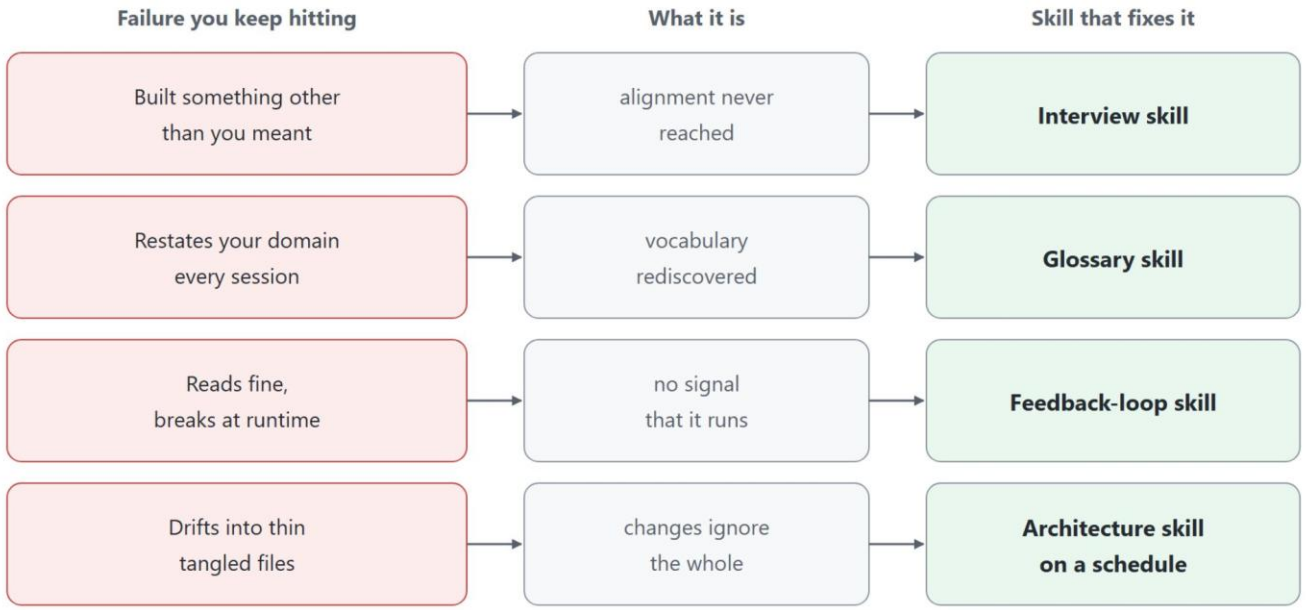
Creating

SKILL.md, frontmatter and a skill from a chat

Starting from a failure mode

The failure you keep paying for names the skill

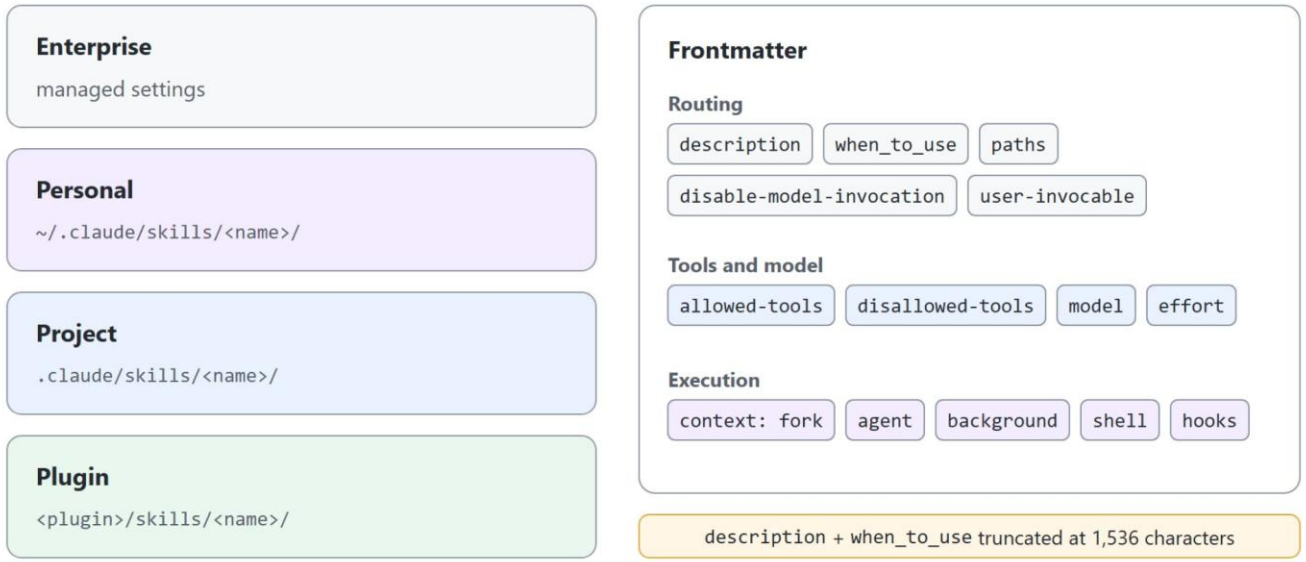
- Built the wrong thing: an **interview** skill that agrees before coding
- Domain re-explained every session: a **glossary** skill
- Reads fine, breaks at runtime: a **feedback-loop** skill, failing test first
- Tangled files: an **architecture** skill run on a schedule
- Write the one you can name from last week



Placement and frontmatter

Where the folder lives and the fields that steer it

- The directory name is the command: `deploy-staging/` gives **`/deploy-staging`**
- Personal `~/.claude/skills/`, project `.claude/skills/`, plugin or enterprise
- Every field is optional; only **`description`** is recommended
- `description` plus `when_to_use` is truncated at **1,536 characters**
- Keep `SKILL.md` under **500 lines**, move detail into linked files



Extracting a skill from a conversation

Prove the workflow first, then encode it

- Work one hard task in a chat until Claude gets it right
- The chat settles the **steps**, the context and the inputs to ask for
- Then ask Claude to turn that sequence into a `SKILL.md`
- Demo: a `sales-by-region.js` module becomes the `data-aggregator` skill



03

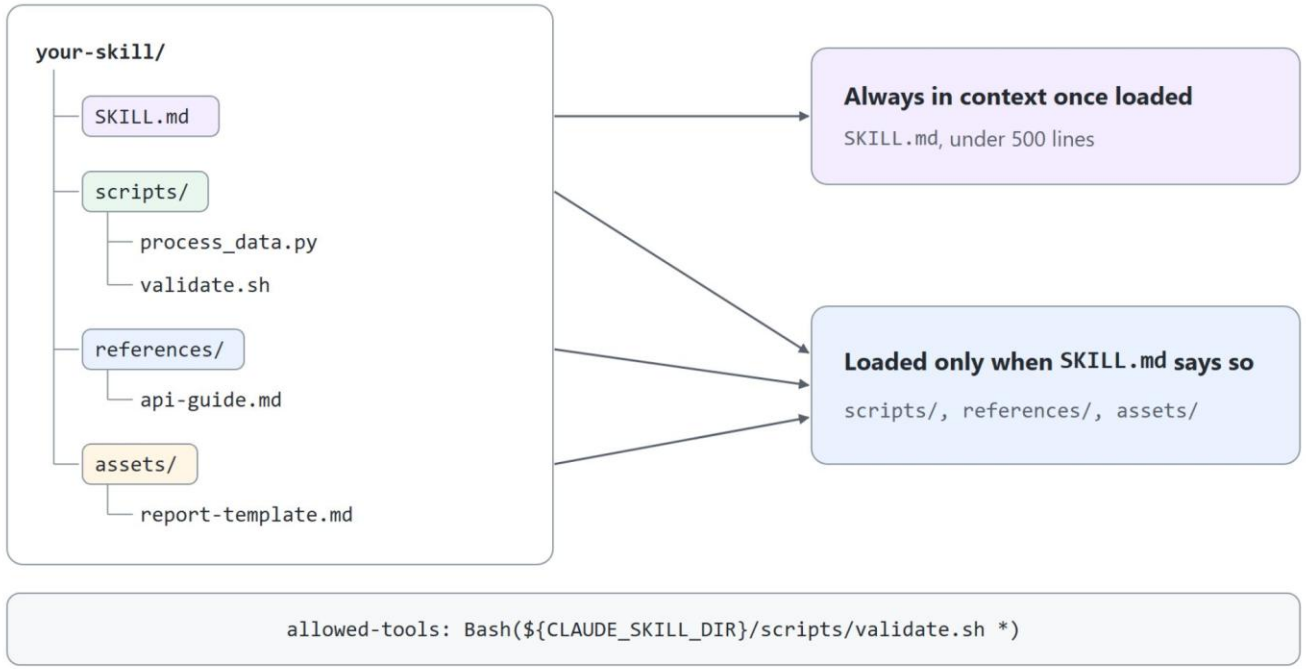
Composite

Scripts, references and composing skills

Composite skill layout

Scripts, references and assets in one portable folder

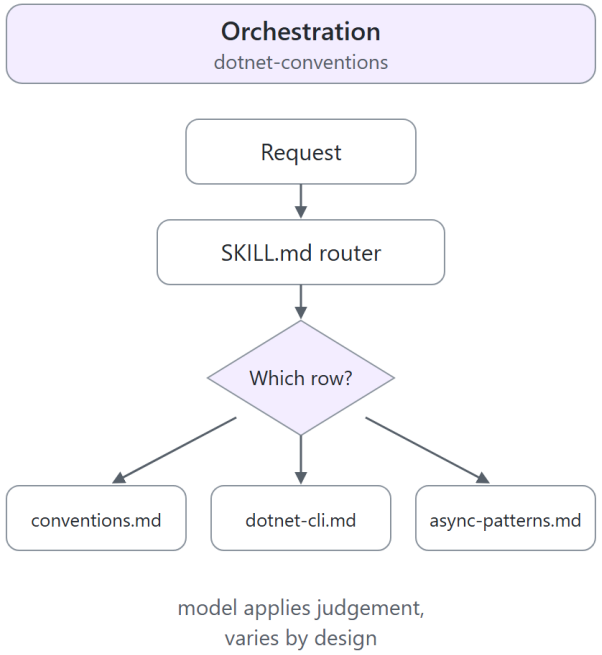
- `SKILL.md` loads on activation and stays for the session
- `scripts/`, `references/`, `assets/` load only when the body says so
- Address scripts through `${CLAUDE_SKILL_DIR}`: commands run from the project root
- The same variable in `allowed-tools` lets the script run without a prompt



Orchestration and deterministic skills

Judgement from Claude, or output from a script

- **Orchestration**: `SKILL.md` holds the procedure, Claude does the work
- **Deterministic**: Claude collects parameters, a bundled script produces the output
- Bundle a script when output must be **exact** or Claude **cannot produce it**
- PNG, PPTX, QR code: a **dry run**, a confirm, then the real run
- Most of a repository's value sits in **orchestration** skills

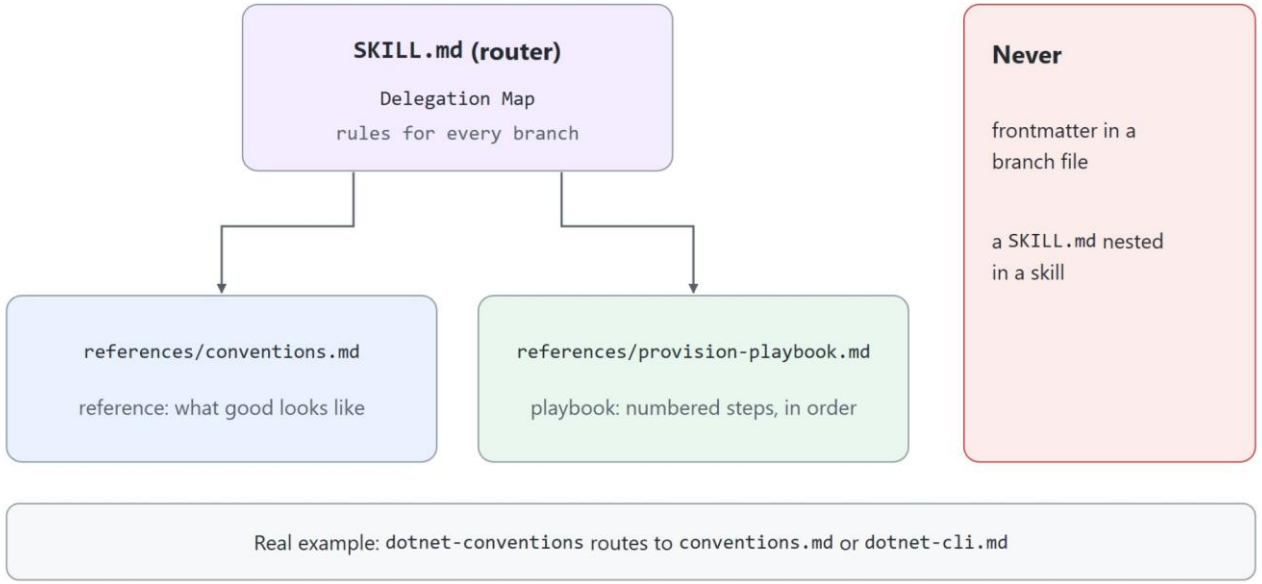




Reference files and playbooks

A router that delegates to branch files

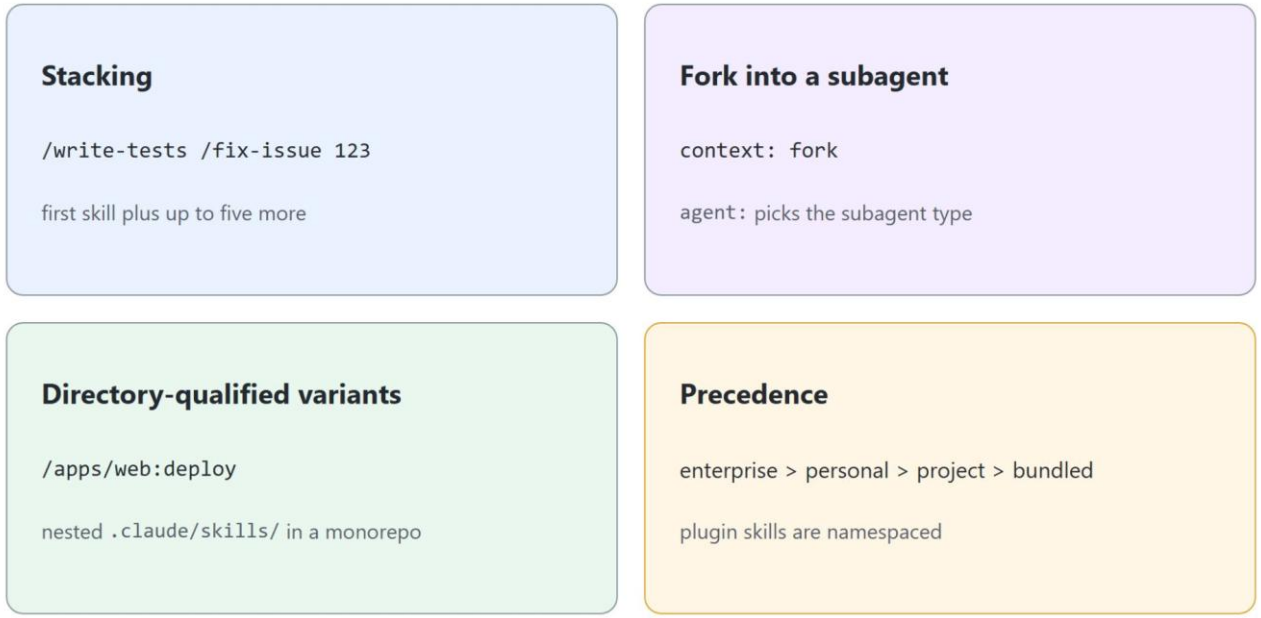
- **Reference** content: knowledge applied to work in progress
- **Playbook** content: an ordered procedure written in the imperative
- `SKILL.md` holds a **delegation map** plus rules that hold for every branch
- Branch files carry **no frontmatter**; never nest a `SKILL.md`
- `dotnet-conventions` loads only the file the request needs



Composing skills

Stacking, forking, variants and precedence

- No `dependencies:` field: composition comes from documented mechanisms
- **Stacking**: `/write-tests /fix-issue 123` loads both with the same arguments
- **`context: fork`** runs a skill as an isolated subagent
- Nested `.claude/skills/` give **`apps/web:deploy`** variants
- Enterprise beats personal beats project; a project skill overrides a bundled one



04

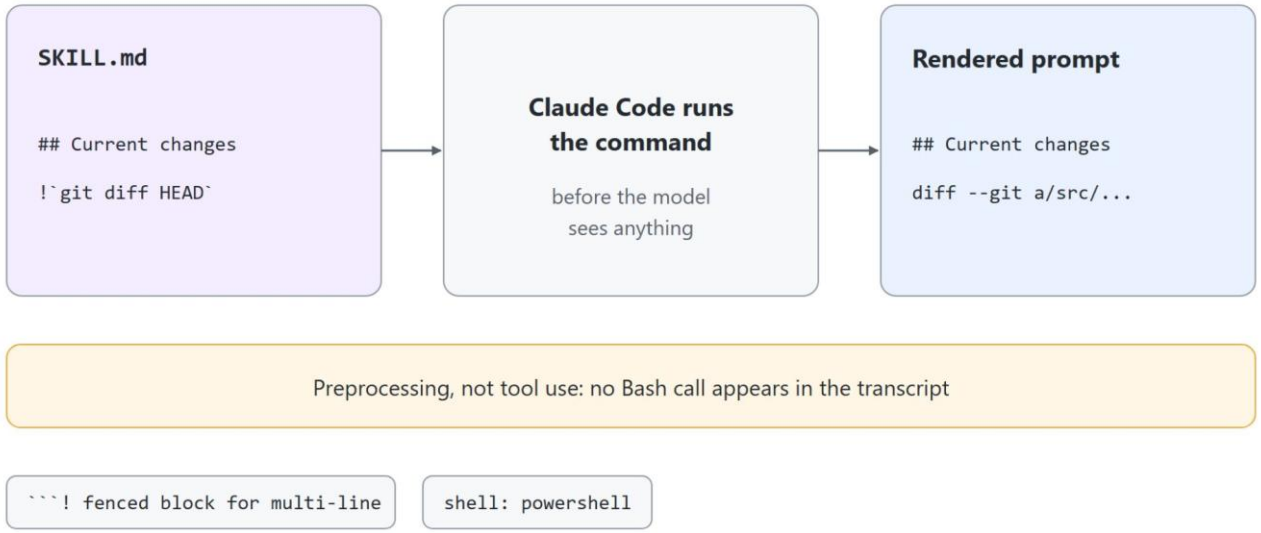
Live Data and Subagents

Injected command output and forked skills

Dynamic context injection

Live command output, inlined before Claude reads

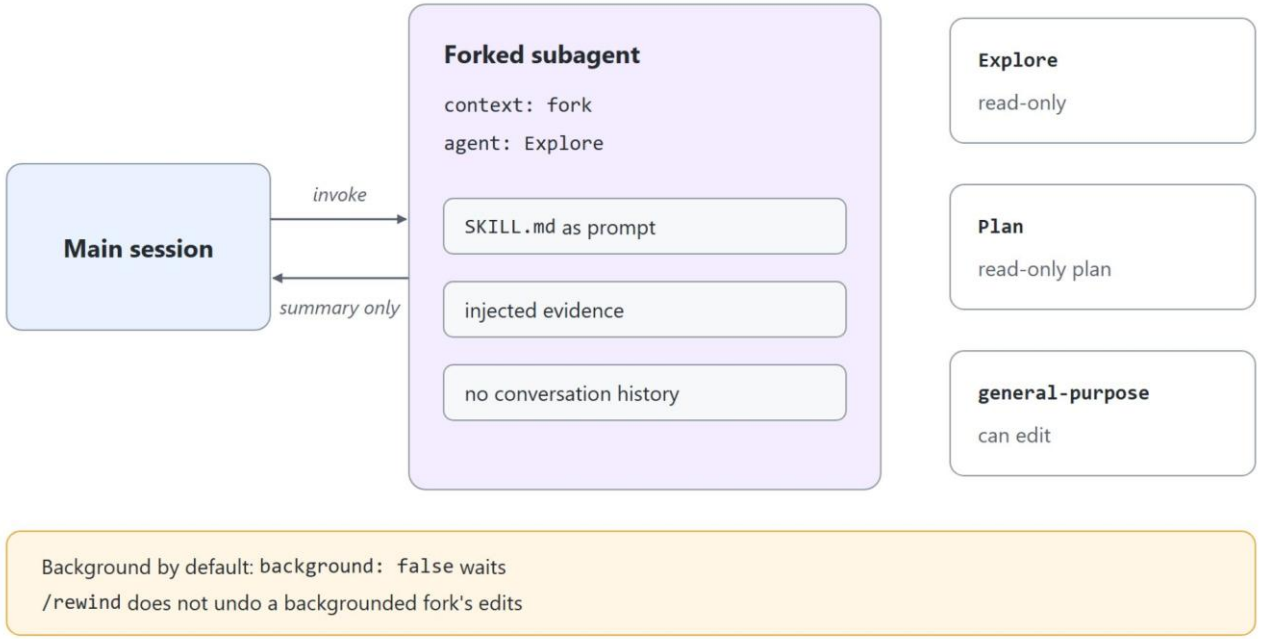
- `!`git diff HEAD`` runs **before** the prompt reaches Claude
- Output replaces the placeholder; no **Bash** call appears in the transcript
- Recognised only at line start or after whitespace; use ```` ```! ```` for several lines
- `shell: powershell` switches the shell; policy can disable it entirely



Running a skill in a subagent

context: fork keeps the reading out of your context

- `context: fork`: the rendered `SKILL.md` becomes the subagent's prompt
- `agent:` picks `Explore`, `Plan`, `general-purpose` or a custom agent
- Runs in the **background** by default; `background: false` waits
- Fork only skills that contain a **task**
- Injection gathers the evidence, the fork spends the tokens elsewhere



05

Test and Package

Trigger tests, four layers and shipping

Measuring whether a skill loads

A headless run instead of a prediction

- Only `description` is in context before a skill loads
- A loaded skill leaves a `Skill` tool call in the stream-json output
- `--disallowedTools` keeps the run from editing; `--allowedTools` does not
- Loading is a model decision: count over several runs, **2 of 3** is a finding
- Test both directions: firing on unrelated work fails as badly as never firing

```
claude -p "Review my changes." --output-format stream-json --verbose
--strict-mcp-config --disallowedTools Edit Write NotebookEdit Bash PowerShell
```

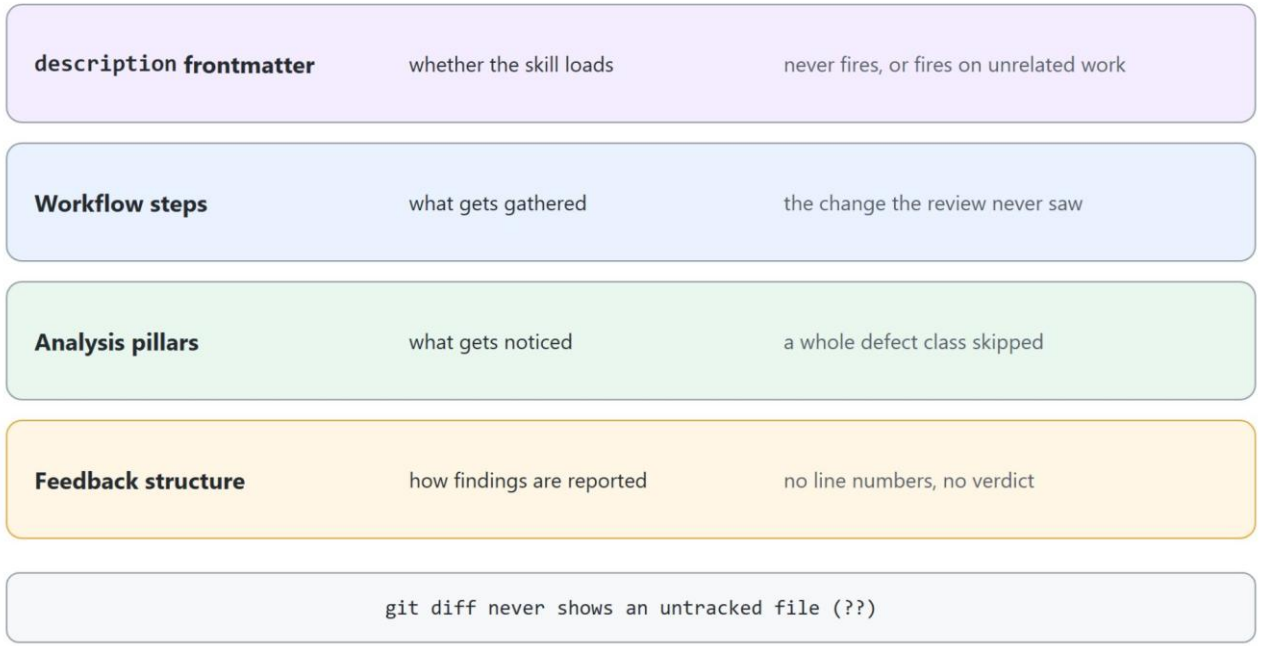
Prompt	Runs	Skill loaded
Review my changes.	3	**3 of 3**
Check this branch before I merge	3	**2 of 3** finding
Rename this variable	3	**0 of 3**, correct

`--allowedTools` auto-approves, it does not restrict

The four layers of a SKILL.md

Every failure belongs to exactly one layer

- **description**: whether the skill loads at all
- **Workflow steps**: what gets gathered
- **Analysis pillars**: what gets noticed
- **Feedback structure**: how findings are reported
- `git diff` never shows an untracked file: the gather step is blind to it



Packaging and distribution

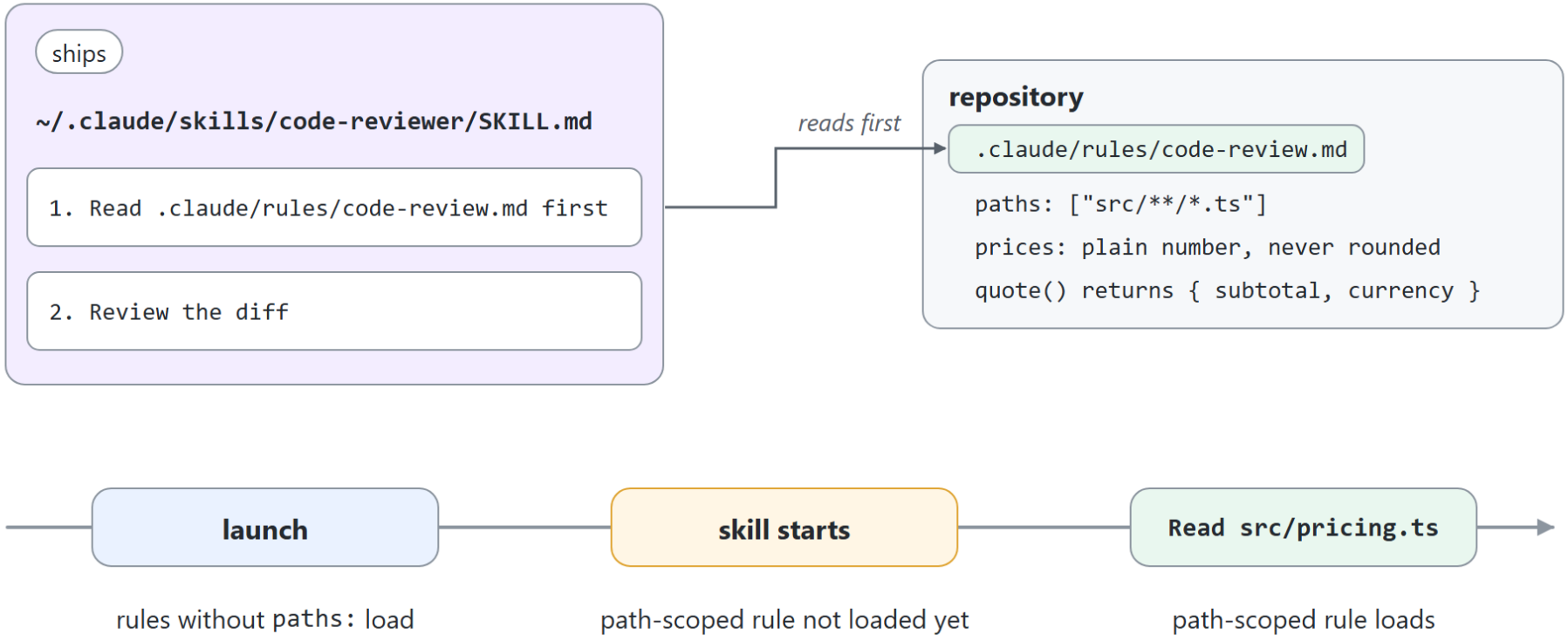
What ships, what stays behind, and how it is pitched

- The package is the **skill folder** and nothing else
- Test matrices, fixtures and the install README stay outside it
- Install via Claude.ai **Settings › Capabilities › Skills** or the skills directory
- API: `container.skills`, with the Code Execution Tool beta
- Pitch the **outcome**, not the folder of YAML and Markdown



Give a global skill your repo's facts

The rules file a skill names is its contract with every repository



The skill reads its rules file first: a path-scoped rule loads only on a matching read

Takeaways

What to carry into the MCP module

- Start from a failure you can name; prove the fix in a chat first
- Write **`description`** with the phrases people actually type
- Split large skills: a router `SKILL.md` and frontmatter-free branch files
- Inject evidence with `` !`command` ``, fork the reading to a subagent
- Measure triggers over several runs and fix the layer that owns the failure

